



CS240 Algorithm Design and Analysis

Lecture 6

Dynamic Programming (Cont.) Network Flow

Qingying Hao

Spring 2026

2026.03.19



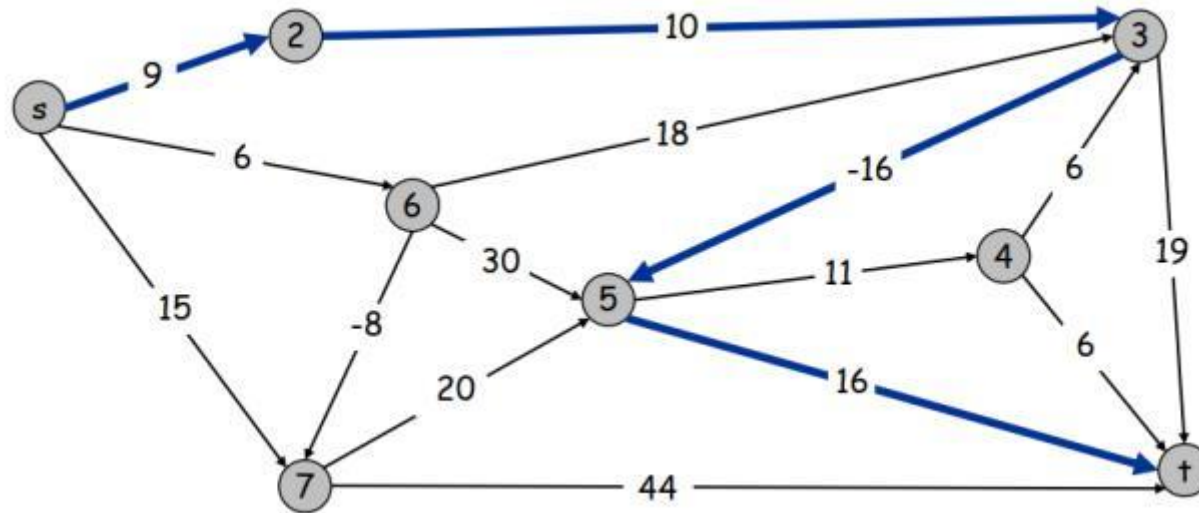
Shortest Paths





Shortest Paths

- Shortest path problem. Given a directed graph $G = (V, E)$, with edge weights c_{vw} , find shortest path from node s to node t
allow negative weights
- Ex. Nodes represent agents in a financial setting and c_{vw} is cost of transaction in which we buy from agent v and sell immediately to w

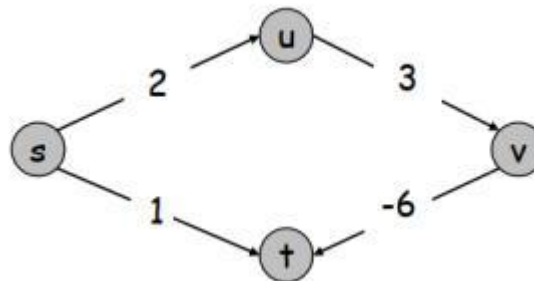




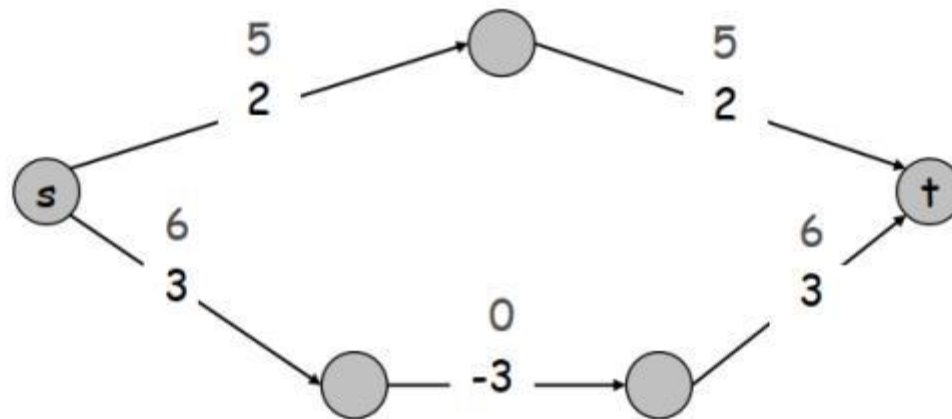
Shortest Paths: Failed Attempts



- **Dijkstra.** Can fail if negative edge costs



- **Re-weighting.** Adding a constant to every edge weight can fail

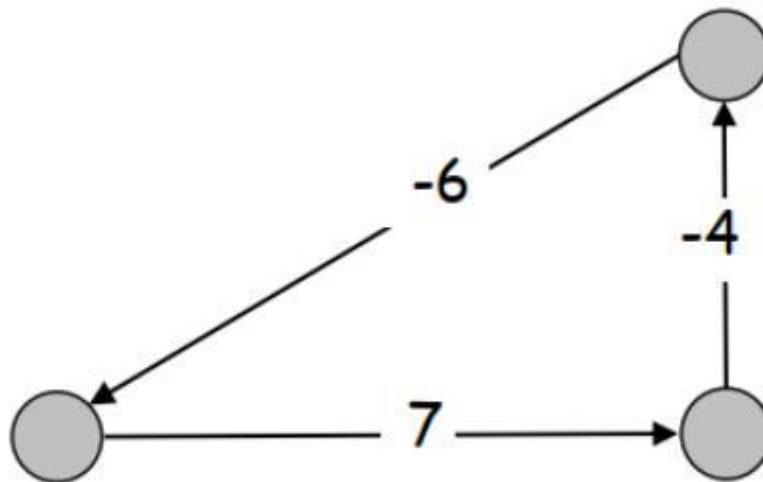




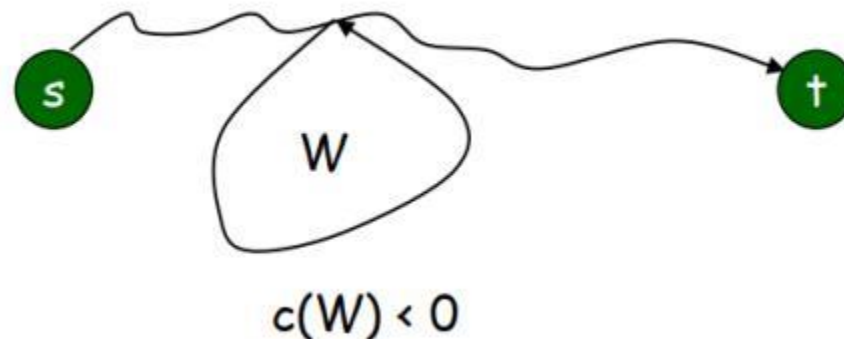
Shortest Paths: Negative Cost Cycles



- Negative cost cycle



- Observation.** If some path from s to t contains a negative cost cycle, there does not exist a shortest s - t path; otherwise, there exists one that is simple





Shortest Paths: Dynamic Programming

- **Def.** $OPT(i, v)$ = length of shortest v - t path P using at most i edges
- **Case 1:** P uses at most $i-1$ edges
 - $OPT(i, v) = OPT(i-1, v)$
- **Case 2:** P may use i edges
 - If (v, w) is first edge, then OPT uses (v, w) , and then selects best w - t path using at most $i-1$ edges

v : current vertex
 w : neighbor

$$OPT(i, v) = \begin{cases} \infty & \text{if } i = 0, v \neq t \\ 0 & \text{if } v = t \\ \min \left\{ OPT(i-1, v), \min_{(v, w) \in E} \{ OPT(i-1, w) + c_{vw} \} \right\} & \text{otherwise} \end{cases}$$

- **Remark.** By previous observation, if no negative cycles, then $OPT(n-1, v)$ = length of shortest v - t path





Shortest Paths: Implementation



```
Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[0, v] \leftarrow \infty$   
     $M[0, t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$   
        foreach node  $v \in V$   
             $M[i, v] \leftarrow M[i-1, v]$   
            foreach edge  $(v, w) \in E$   
                 $M[i, v] \leftarrow \min \{ M[i, v], M[i-1, w] + c_{vw} \}$   
  
    return  $M[n-1, s]$   
}
```

Analysis. $\Theta(mn)$ time, $\Theta(n^2)$ space

Finding the shortest paths. Maintain a “successor” for each table entry





Shortest Paths: Improvements



```
Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[0, v] \leftarrow \infty$   
     $M[0, t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$   
        foreach node  $v \in V$   
             $M[i, v] \leftarrow M[i-1, v]$   
            foreach edge  $(v, w) \in E$   
                 $M[i, v] \leftarrow \min \{ M[i, v], M[i-1, w] + c_{vw} \}$   
  
    return  $M[n-1, s]$   
}
```

Practical improvements.

- Maintain only one array $M[v]$ = shortest v - t path that we have found so far





Shortest Paths: Improvements



```
Shortest-Path( $G, s, t$ ) {  
  foreach node  $v \in V$   
     $M[v] \leftarrow \infty$   
   $M[t] \leftarrow 0$   
  
  for  $i = 1$  to  $n-1$   
    foreach node  $v \in V$   
     $M[v] \leftarrow M[v]$   
    foreach edge  $(v, w) \in E$   
       $M[v] \leftarrow \min \{ M[v], M[w] + c_{vw} \}$   
  
  return  $M[s]$   
}
```

Practical improvements.

- Maintain only one array $M[v]$ = shortest v - t path that we have found so far
- No need to check edges of the form (v, w) unless $M[w]$ changed in previous iteration





Bellman-Ford: Efficient Implementation



```
Push-Based-Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[v] \leftarrow \infty$   
     $M[t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$  {  
        foreach node  $w \in V$   
            if ( $M[w]$  has been updated in previous iteration)  
                foreach node  $v$  such that  $(v, w) \in E$   
                    if ( $M[v] > M[w] + c_{vw}$ )  
                         $M[v] \leftarrow M[w] + c_{vw}$   
        If no  $M[w]$  value changed in iteration  $i$ , stop.  
    }  
  
    return  $M[s]$   
}
```

Analysis.

- $O(n)$ extra space
- Time: $O(mn)$ worst case, but substantially faster in practice





Application 1: Distance Vector Protocol





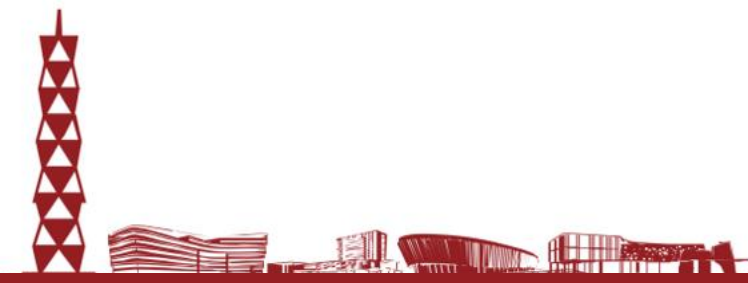
Distance Vector Protocol



- **Communication network**

- Nodes $\approx$ routers
- Edges $\approx$ direct communication link
- Cost of edge $\approx$ delay on link $\longleftarrow$ naturally nonnegative

- **Dijkstra's algorithm.** Requires global information of network
- **Bellman-Ford.** Uses only local knowledge of neighboring nodes
- **Synchronization.** We don't expect routers to run in lockstep. The order in which each foreach loop executes is not important. Moreover, algorithm still converges even if updates are asynchronous

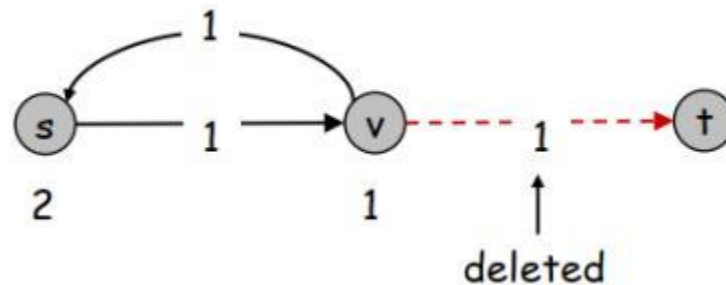




Distance Vector Protocol

- **Distance vector protocol**

- Each router maintains a vector of shortest path lengths to every other node (distances) and the first hop on each path (directions)
 - Algorithm: each router performs n separate computations, one for each potential destination node
 - “Routing by rumor”
-
- **Ex.** RIP, Xerox XNS RIP, Novell’s IPX RIP, Cisco’s IGRP, DEC’s DNA Phase IV, AppleTalk’s RTMP
-
- **Caveat.** Edge costs may change during algorithm (or fail completely)



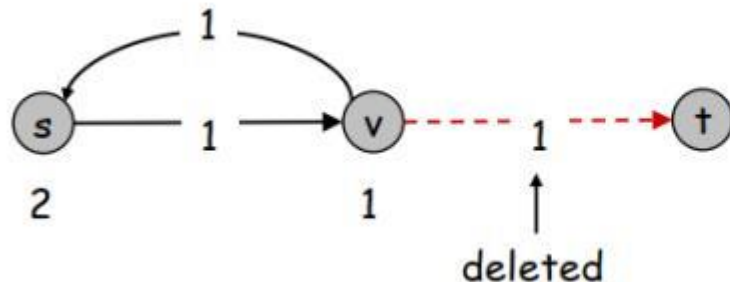
"counting to infinity"





Distance Vector Protocol

- **Caveat.** Edge costs may change during algorithm (or fail completely)



"counting to infinity"

Step 1: Initial (Correct State)

Each node knows the shortest distance to **t**:

$s \text{ --- } v \text{ --- } t$

$\text{Cost}(s \rightarrow v) = 1$

$\text{Cost}(v \rightarrow t) = 1$

$\text{Cost}(s \rightarrow t) = 2$

Step 2: Link Failure

V-t link deleted

v can no longer reach t

Step 3: Problem Begins

v detects the failure $\rightarrow$ sets distance to $t = \infty$

Step 4: Wrong Updates

s says: distance to $t = 2$

So v thinks it can reach t via 2

$v \rightarrow s \rightarrow t$ at $2+1 = 3$

$\text{distance}(v, t) = 3$ ❌

So s hears v says distance = 3

$s \rightarrow v \rightarrow t = 1 + 3 = 4$

$\text{distance}(s, t) = 4$ ❌





Path Vector Protocols



- **Link state routing**

- Each router also stores the entire path
- Avoids “counting-to-infinity” problem and related difficulties
- Requires significantly more storage

← not just the distance and first hop

- **Ex.** Border Gateway Protocol (BGP), Open Shortest Path First (OSPF)





Application 2: Negative Cycles in a Graph

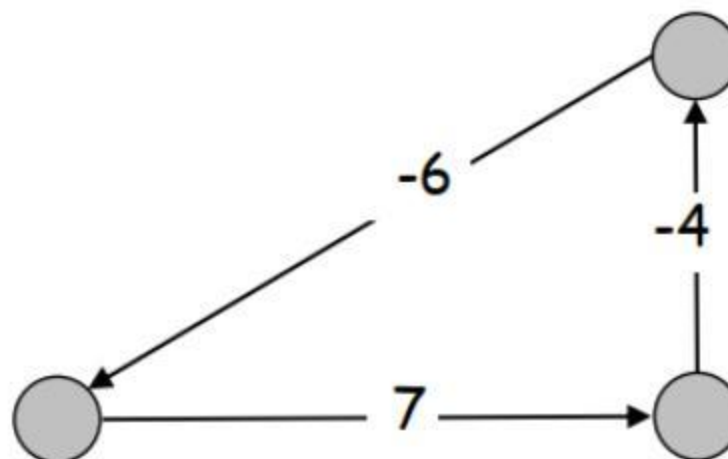




Detecting Negative Cycles



- **Negative cycle detection problem.** Given a digraph $G = (V, E)$, with edge weights c_{vw} , find a negative cycle (if one exists)



Detecting Negative Cycles: Application

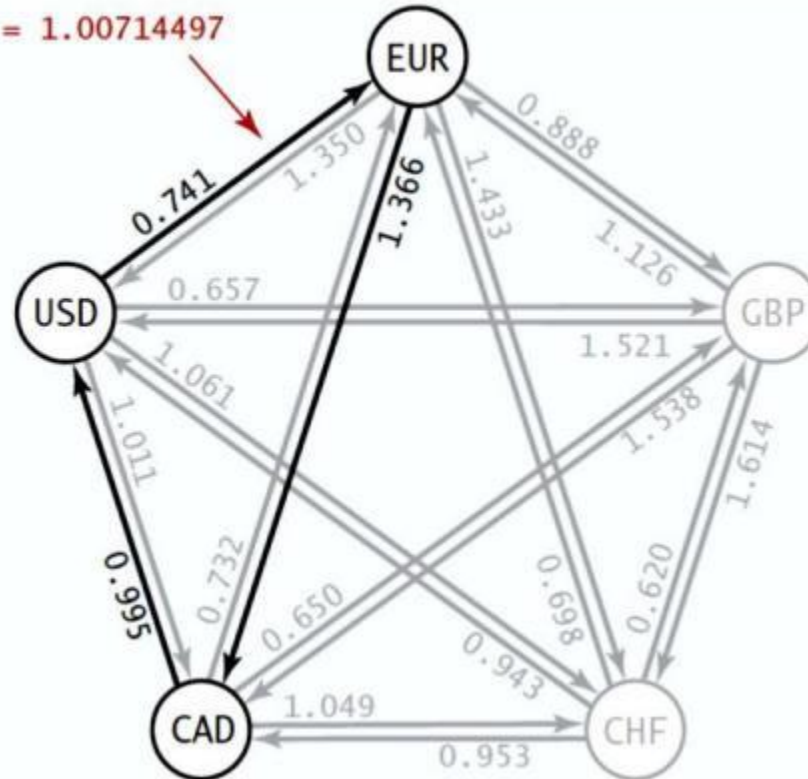
- **Currency conversion.** Given n currencies and exchange rates between pairs of currencies, is there an arbitrage opportunity?
- **Remark.** Fastest algorithm very valuable!

$$0.741 * 1.366 * .995 = 1.00714497$$

$$w_i = -\log(r_i)$$

$$\begin{aligned} w_1 + w_2 + \dots + w_k \\ &= -\log(r_1) - \log(r_2) - \dots - \log(r_k) \\ &= -\log(r_1 \times r_2 \times \dots \times r_k) \end{aligned}$$

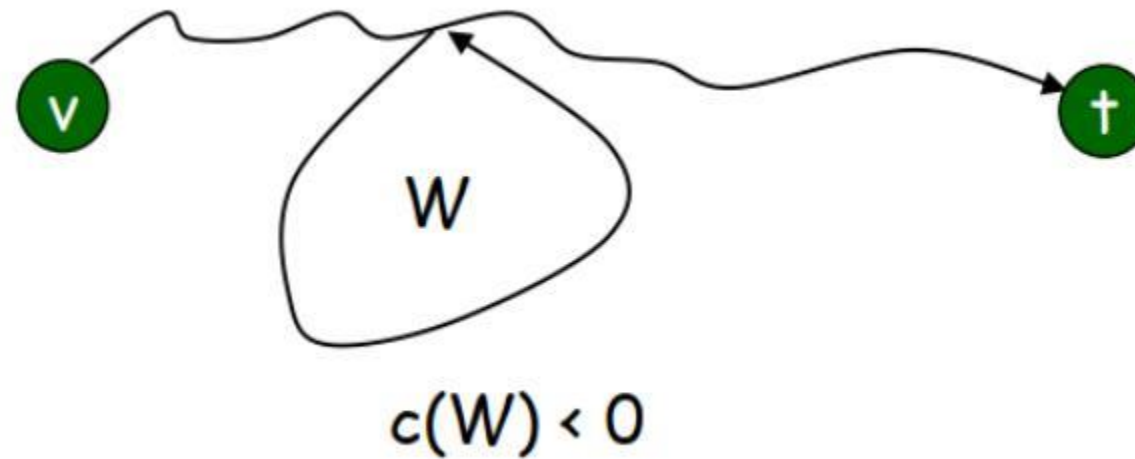
$$\begin{aligned} -\log(\text{product}) &< 0 \\ \text{product} &> 1 \end{aligned}$$





Detecting Negative Cycles

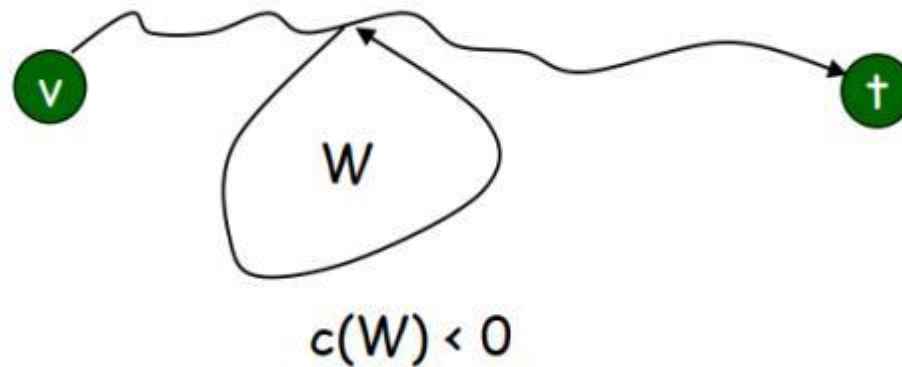
- **Lemma.** If $\text{OPT}(n, v) = \text{OPT}(n-1, v)$ for all v , then there is no negative cycle with a path to t
- **Pf.** (by contradiction)
 - $\text{OPT}(n, v) = \text{OPT}(n-1, v) \rightarrow \text{OPT}(i, v) = \text{OPT}(n-1, v)$ for $i \geq n$
 - But negative cycle in a path implies that $\text{OPT}(i, v)$ always decreases as i increases





Detecting Negative Cycles

- **Lemma.** If $\text{OPT}(n, v) < \text{OPT}(n-1, v)$ for some node v , then (any) shortest path from v to t contains a cycle W . Moreover, W has negative cost
- **Pf.** (by contradiction)
 - Since $\text{OPT}(n, v) < \text{OPT}(n-1, v)$, we know the shortest v - t path P has exactly n edges
 - By pigeonhole principle, P must contain a directed cycle W
 - Deleting W yields a v - t path with $< n$ edges $\rightarrow W$ has negative cost

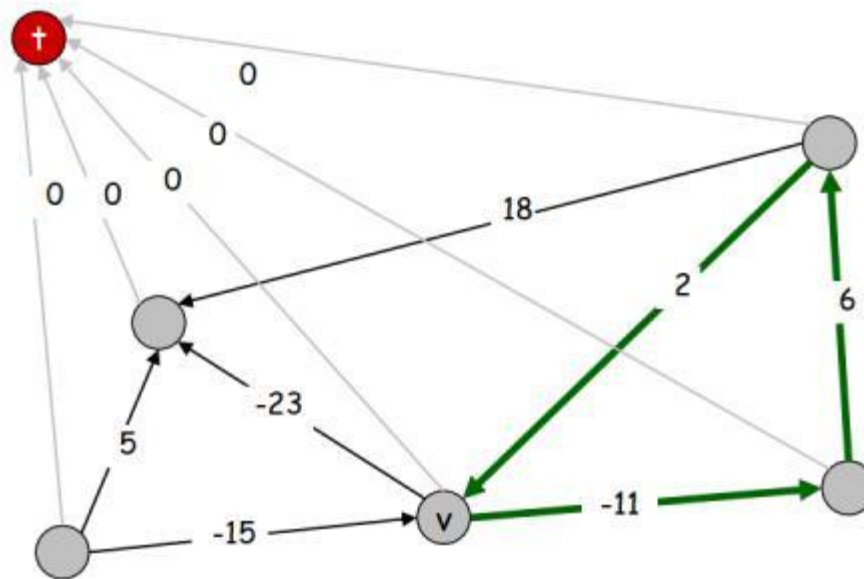




Detecting Negative Cycles

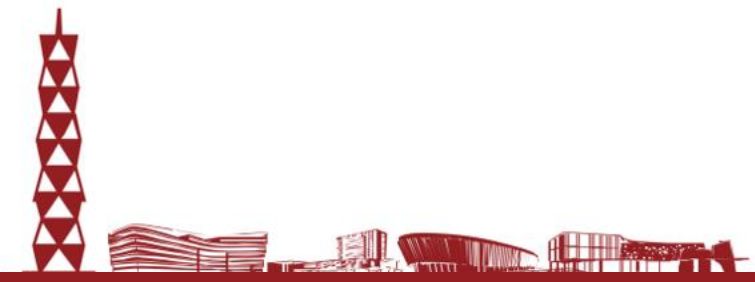


- **Theorem.** Can detect negative cost cycle in $O(mn)$ time
 - Add new node t and connect all nodes to t with 0-cost edge
 - Check if $\text{OPT}(n, v) = \text{OPT}(n-1, v)$ for all nodes v
 - If yes, then no negative cycles
 - If no, then extract cycle from shortest path from v to t





Dynamic Programming: Summary





Dynamic Programming

- **Basic idea**

- Polynomial number of sub-problems with a natural ordering from smallest to largest
- Optimal solution to a sub-problem can be constructed from optimal solutions of smaller sub-problems
- Sub-problems are overlapping!

- **Guideline**

- Define the sub-problems
 - $OPT(\dots)$
- Write down the recursive formulas
 - Ex: $OPT(i) = \max(f(OPT(j)), g(OPT(k)), \dots), j, k < i$
- Compute the formulas either bottom-up or top-down





Dynamic Programming



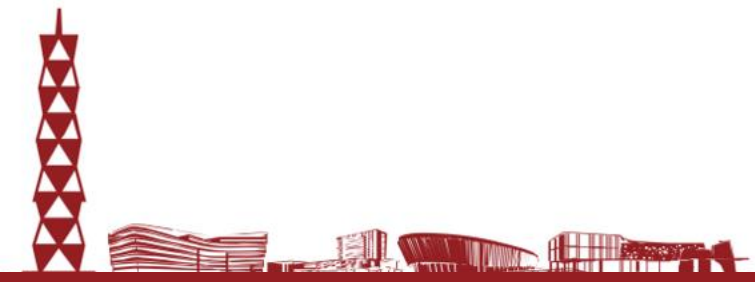
- **Algorithms**

- Weighted interval scheduling
 - 1D array; binary choice
- Knapsack
 - 2D array; adding a new variable (weight limit)
- RNA secondary structure
 - 2D array: intervals
- Sequence Alignment
 - 2D array: prefix alignment
- Sequence Alignment in Linear Space
 - Combination of divide-and-conquer and dynamic programming
- Shortest path with negative edges
 - (Bellman-Ford) 2D array: shortest path with edge number $\leq i$
- Distance Vector Protocol
- Negative Cycle Detection





Network Flow





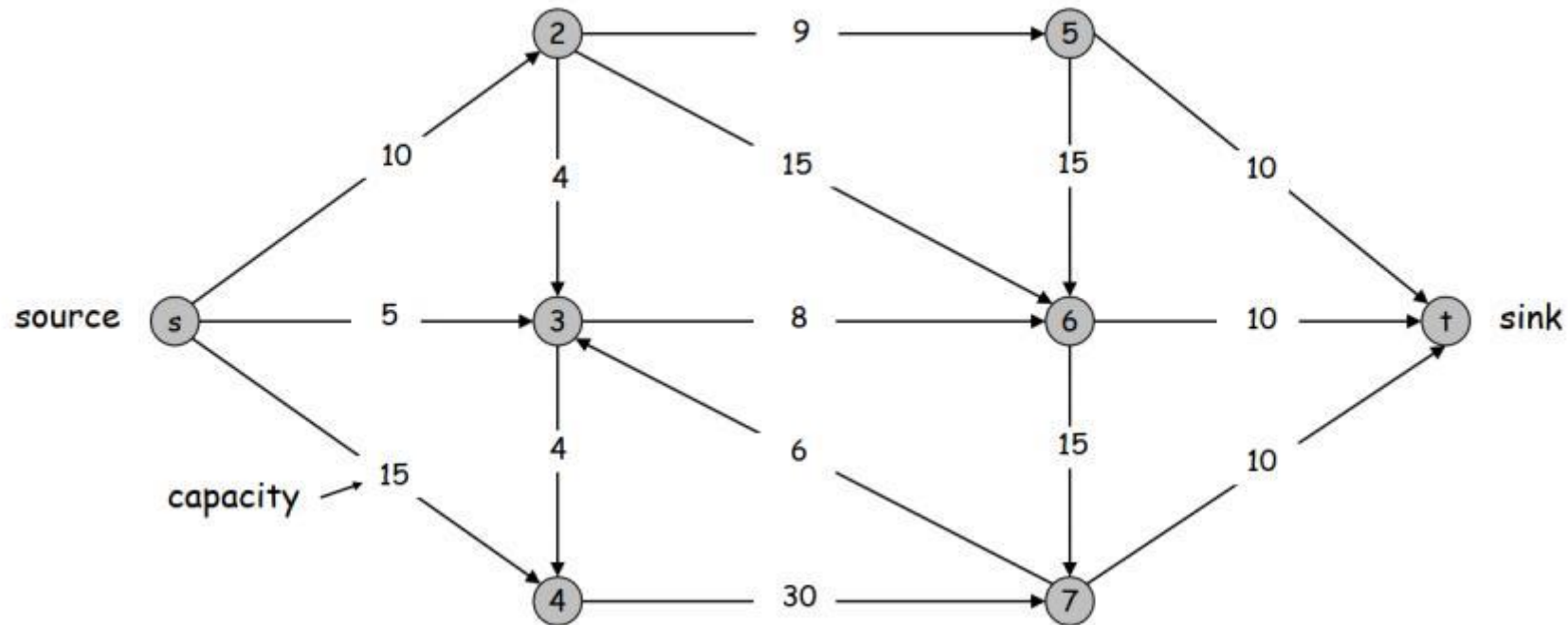
Max-flow and Ford-Fulkerson Algorithm





Flows

- Flow network
 - Abstraction for material **flowing** through the edges
 - $G = (V, E)$ = directed graph
 - Two distinguished nodes: s = source, t = sink
 - $c(e)$ = nonnegative capacity of edge e

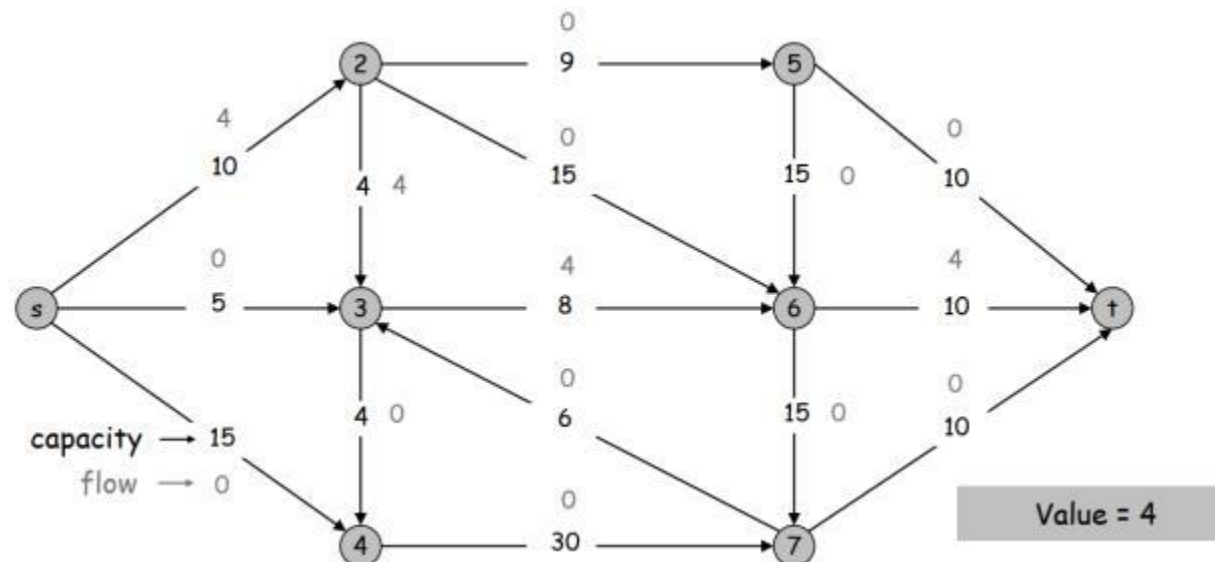




Flows

- **Def.** An **s-t flow** is a function that satisfies:
 - For each $e \in E$: $0 \leq f(e) \leq c(e)$ (capacity)
 - For each $v \in V - \{s, t\}$: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ (flow conservation)

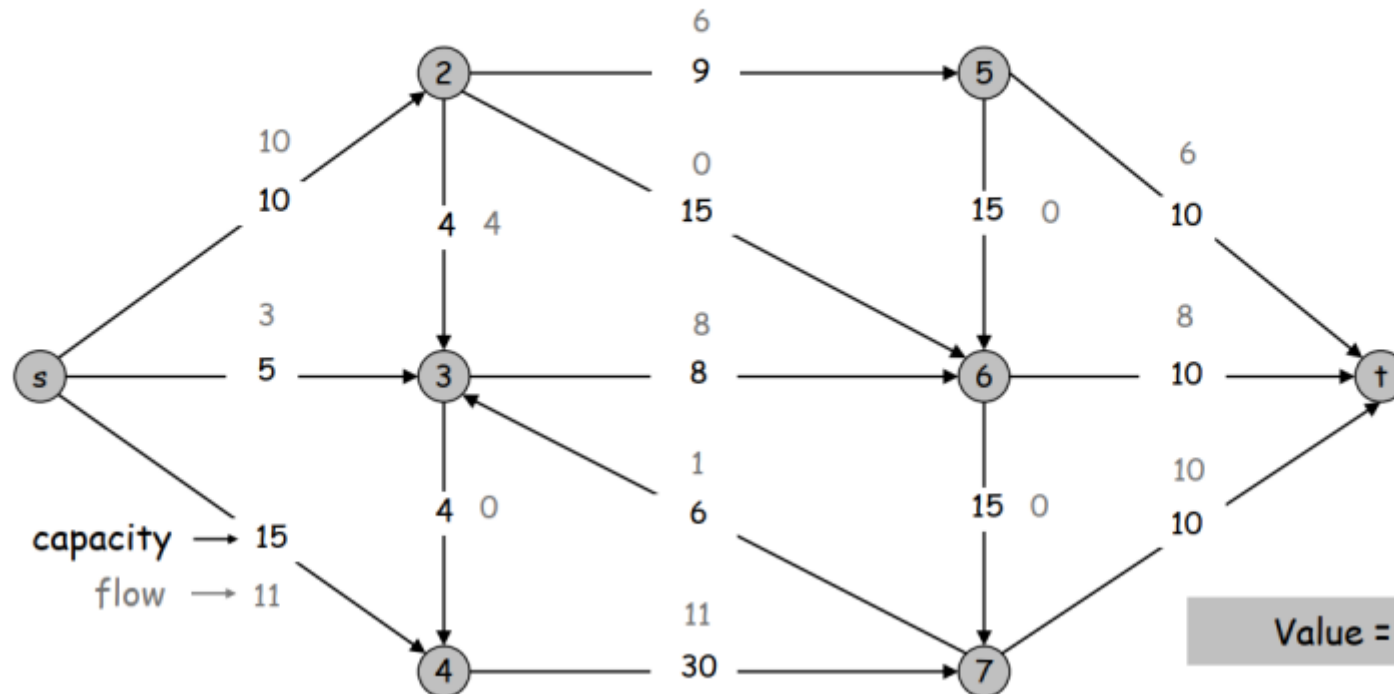
- **Def.** The value of a flow f is : $v(f) = \sum_{e \text{ out of } s} f(e)$





Flows

- **Def.** An **s-t flow** is a function that satisfies:
 - For each $e \in E$: $0 \leq f(e) \leq c(e)$ (capacity)
 - For each $v \in V - \{s, t\}$: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ (flow conservation)
- **Def.** The value of a flow f is : $v(f) = \sum_{e \text{ out of } s} f(e)$

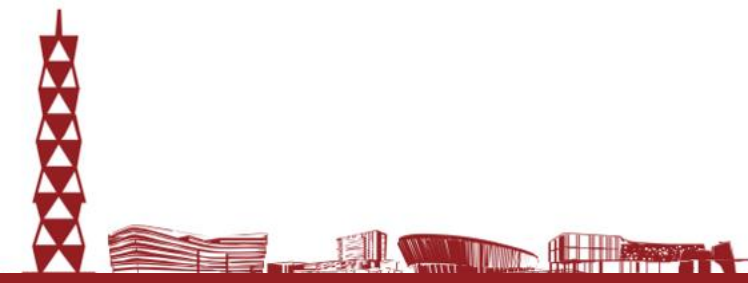
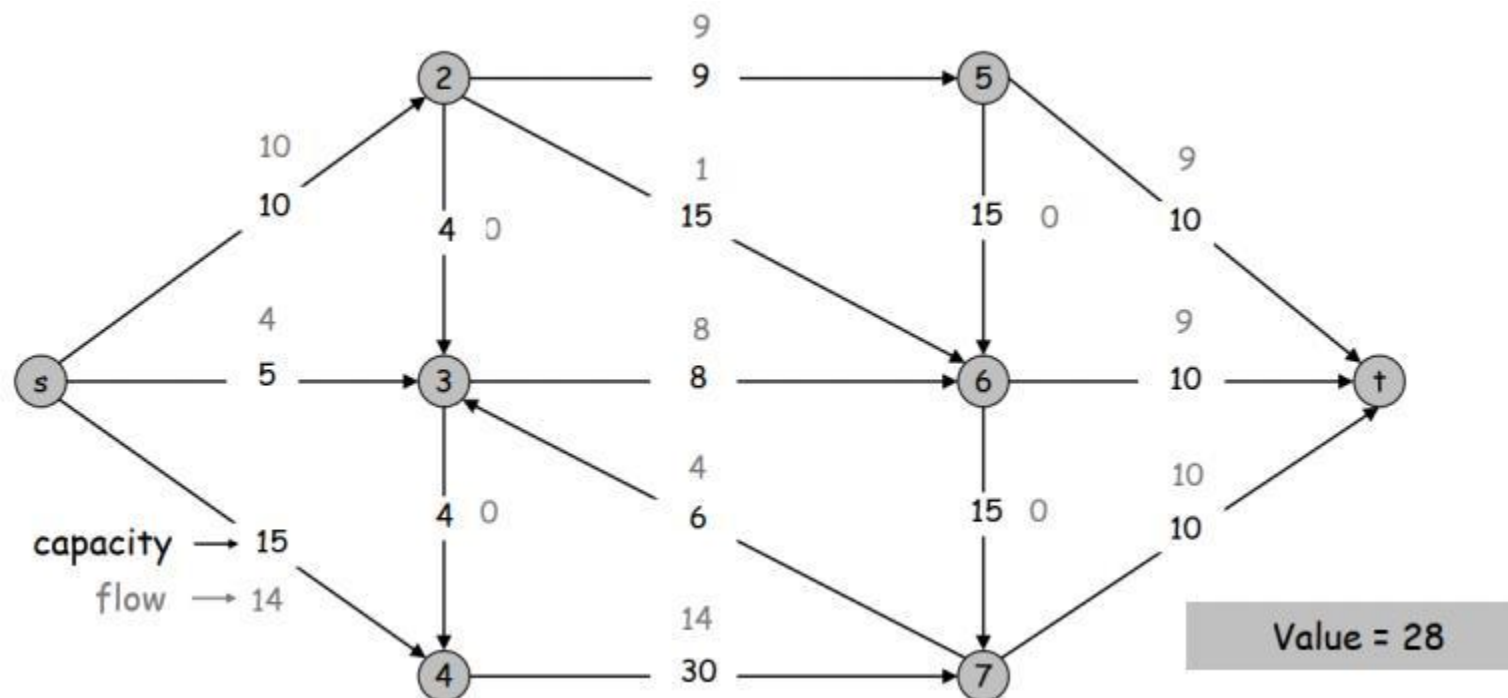




Maximum Flow Problem



- **Max flow problem.** Find s-t flow of maximum value



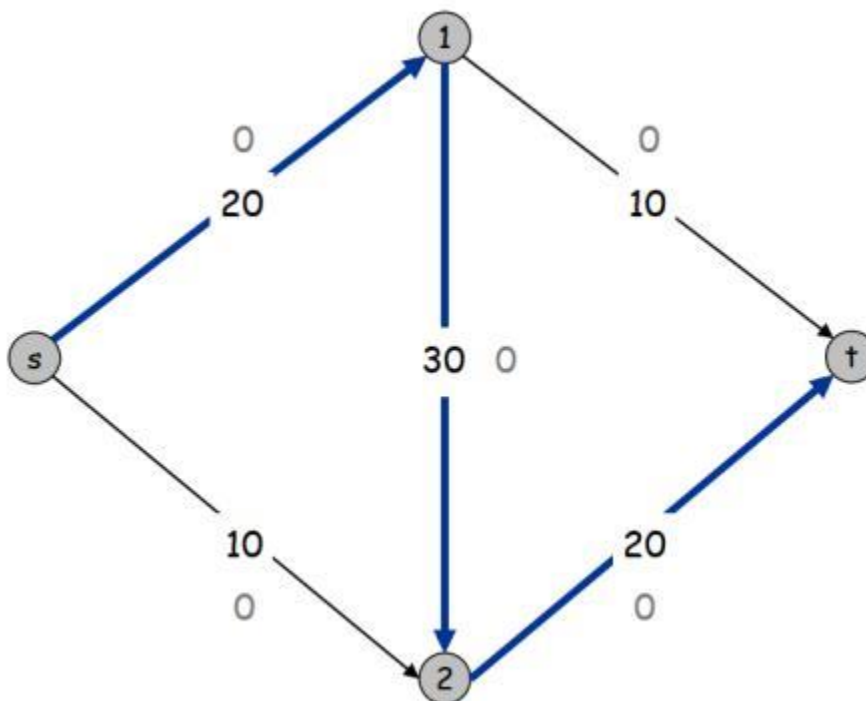


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 0



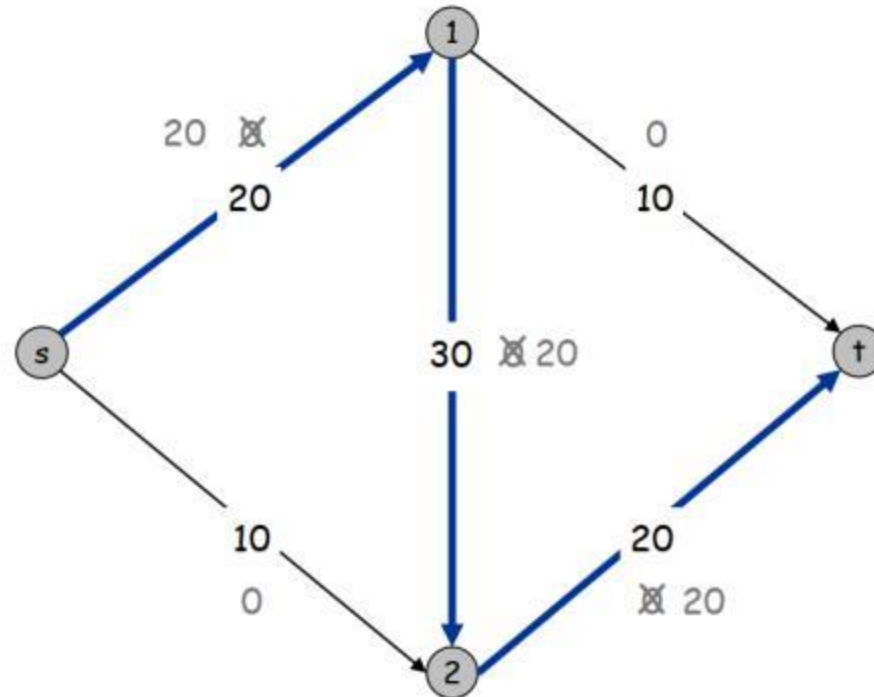


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 20



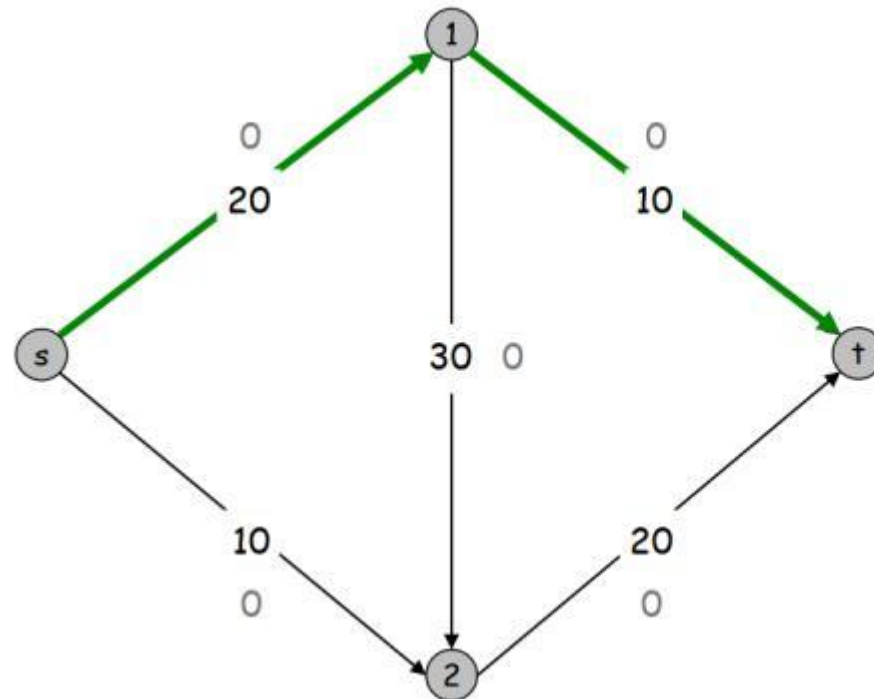


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 0



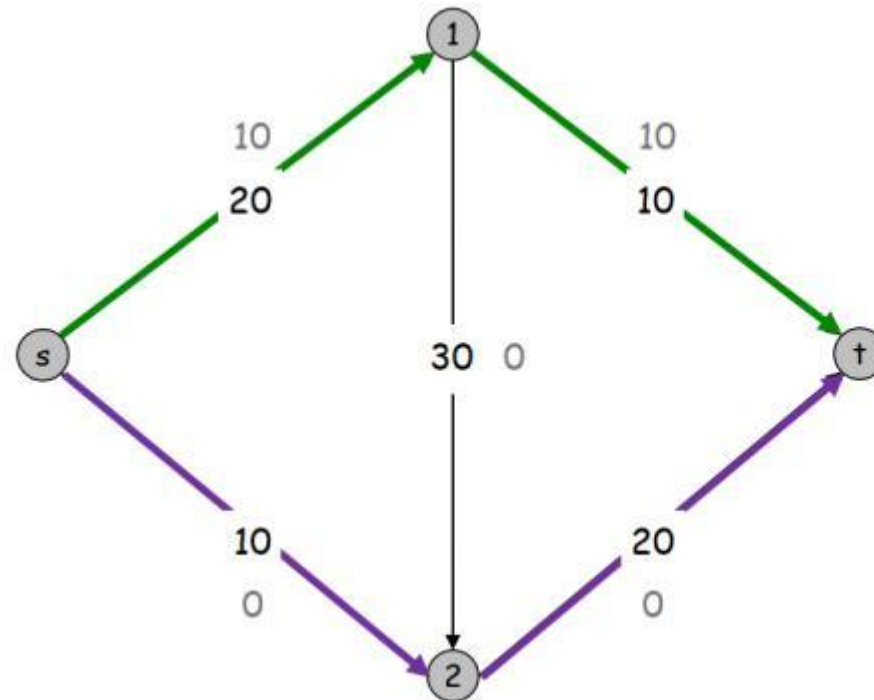


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 10



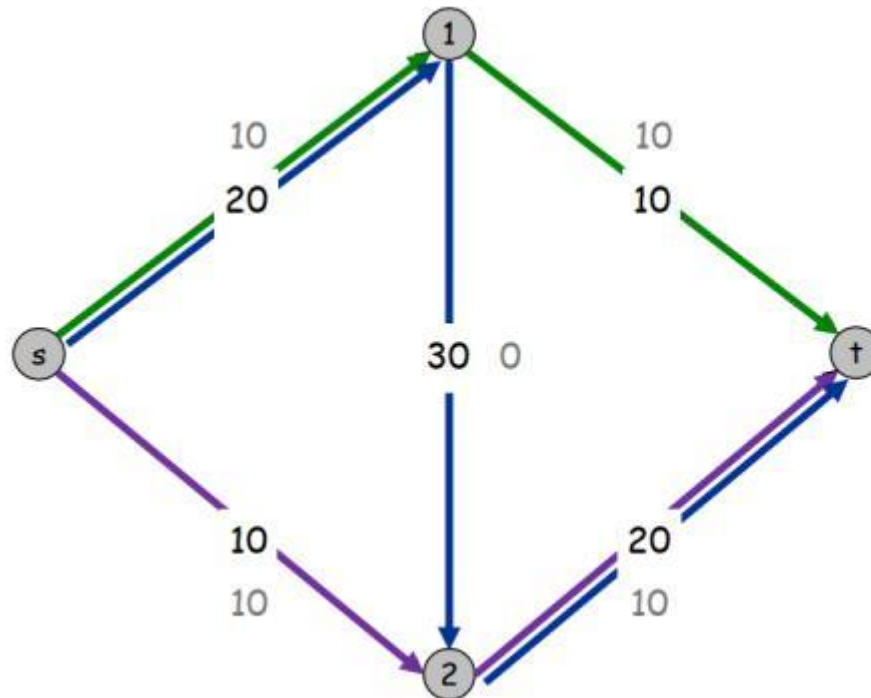


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 20



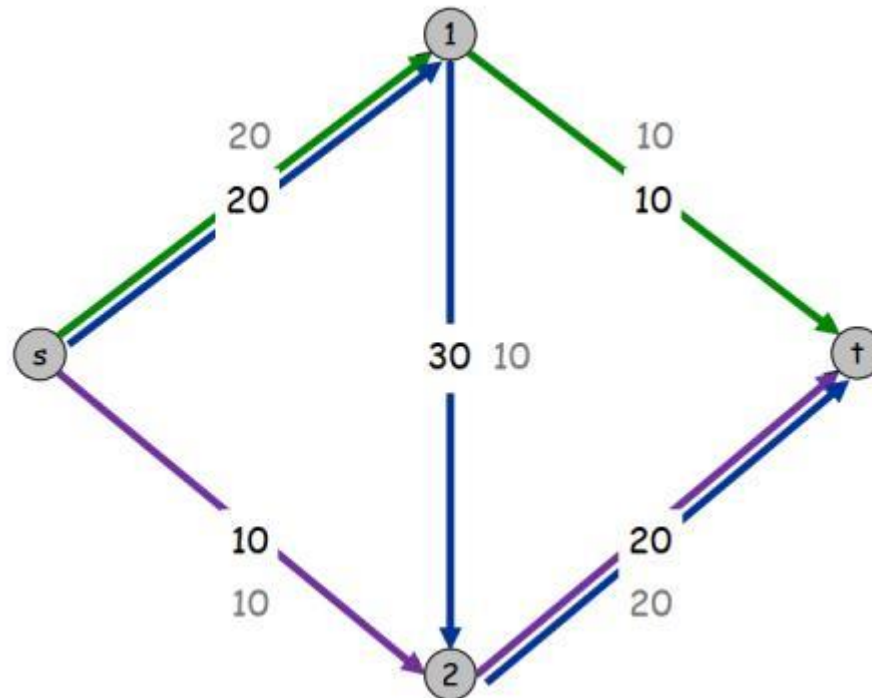


Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
- Repeat until you get stuck



Flow value = 30





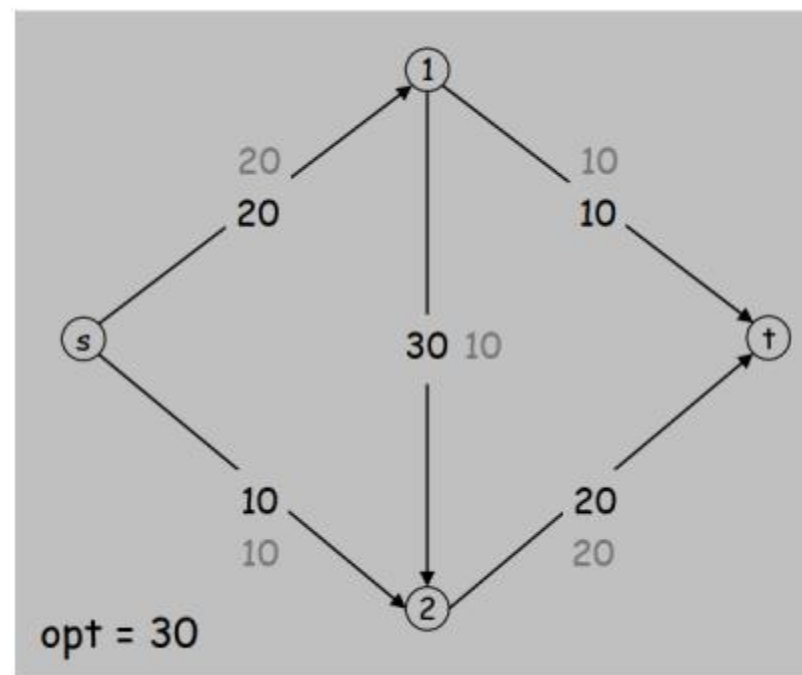
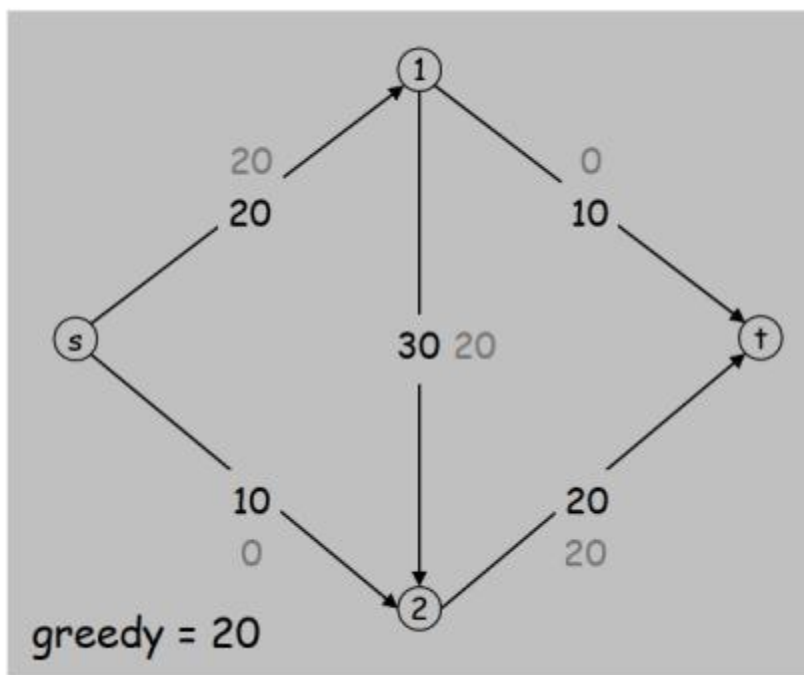
Towards a Max Flow Algorithm



- **Greedy algorithm**

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an s-t path P where each edge has $f(e) < c(e)$
- Augment flow along path P
 - Repeat until you get **stuck**

locally optimality $\neq$ globally optimality

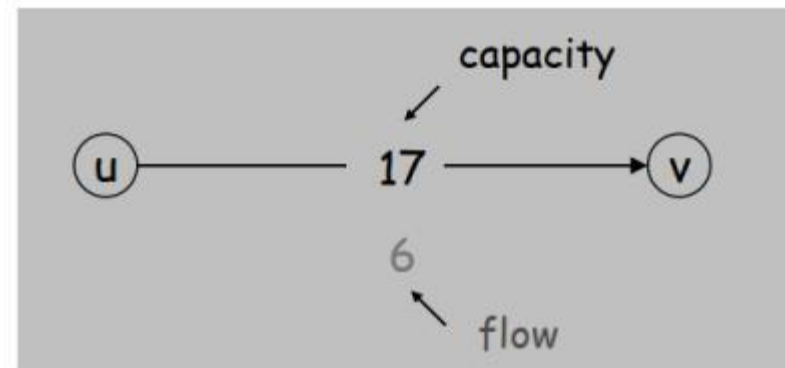




Residual Graph

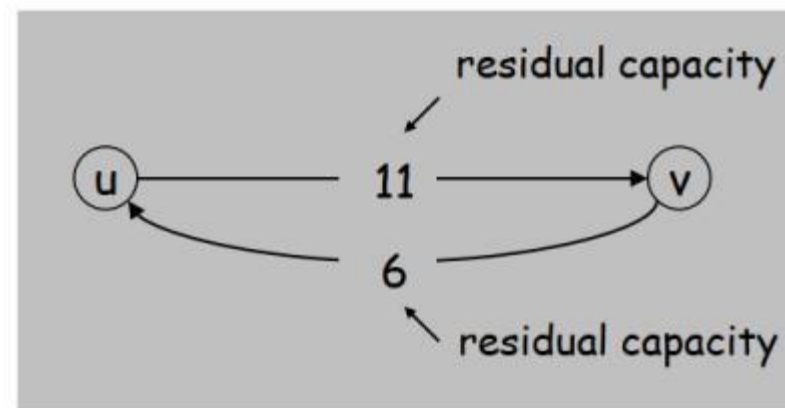


- **Original edge:** $e = (u, v) \in E$
 - Flow $f(e)$, capacity $c(e)$



- **Residual edge**
 - “Undo” flow sent
 - $e = (u, v)$ and $e^R = (v, u)$
 - Residual capacity:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E \\ f(e) & \text{if } e^R \in E \end{cases}$$



- **Residual graph:** $G_f = (V, E_f)$
 - Residual edges with positive residual capacity
 - $E_f = \{e: f(e) < c(e)\} \cup \{e^R: f(e) > 0\}$





Augmenting Path

- **Augmenting path:** a simple s-t path P in the residual graph G_f
- **Bottleneck capacity:** for an augmenting path P is the minimum residual capacity of any edge in P

```
Augment(f, c, P) {  
    b ← bottleneck(P)  
    foreach e ∈ P {  
        if (e ∈ E) f(e) ← f(e) + b  
        else      f(eR) ← f(eR) - b  
    }  
    return f  
}
```

forward edge

reverse edge

- **Claim:** After augmentation, f is still a flow





Ford-Fulkerson Algorithm



- **Ford-Fulkerson Algorithm**

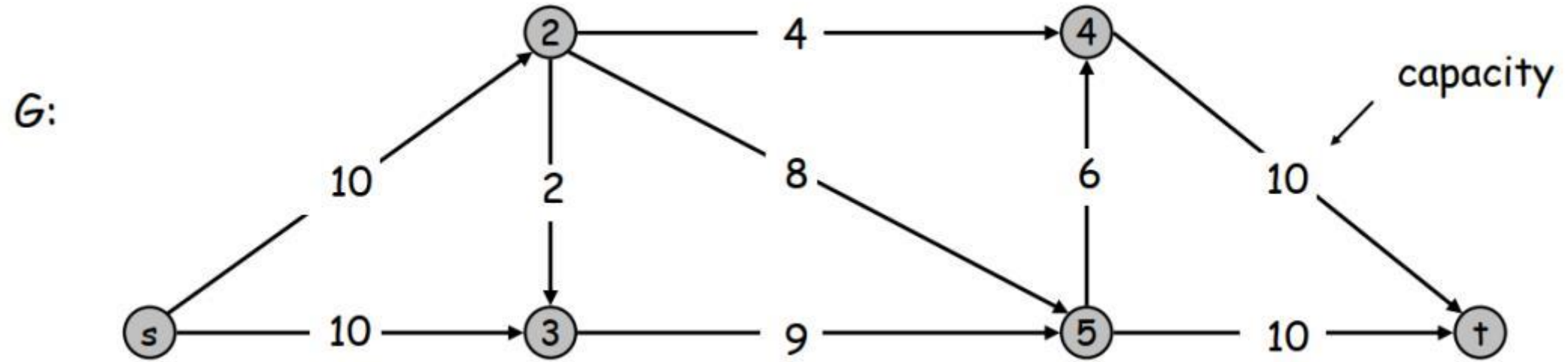
- Start with $f(e) = 0$ for all edge $e \in E$
- Find an augmenting path P in the residual graph G_f
- Augment flow along path P
- Repeat until you get stuck

```
Ford-Fulkerson( $G, s, t, c$ ) {  
    foreach  $e \in E$   $f(e) \leftarrow 0$   
     $G_f \leftarrow$  residual graph  
  
    while (there exists augmenting path  $P$ ) {  
         $f \leftarrow$  Augment( $f, c, P$ )  
        update  $G_f$   
    }  
    return  $f$   
}
```



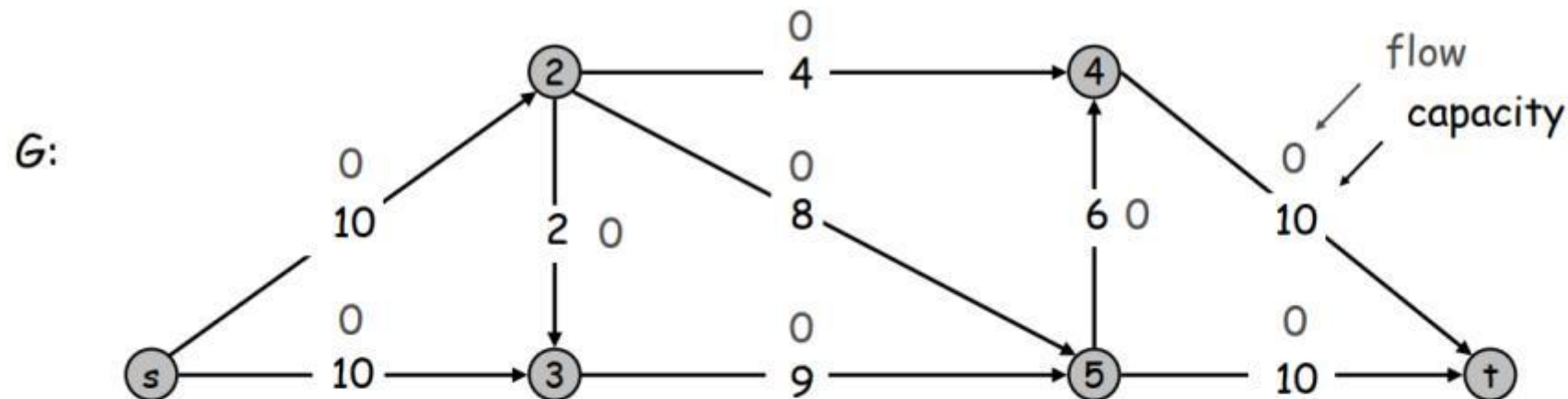


Ford-Fulkerson Algorithm

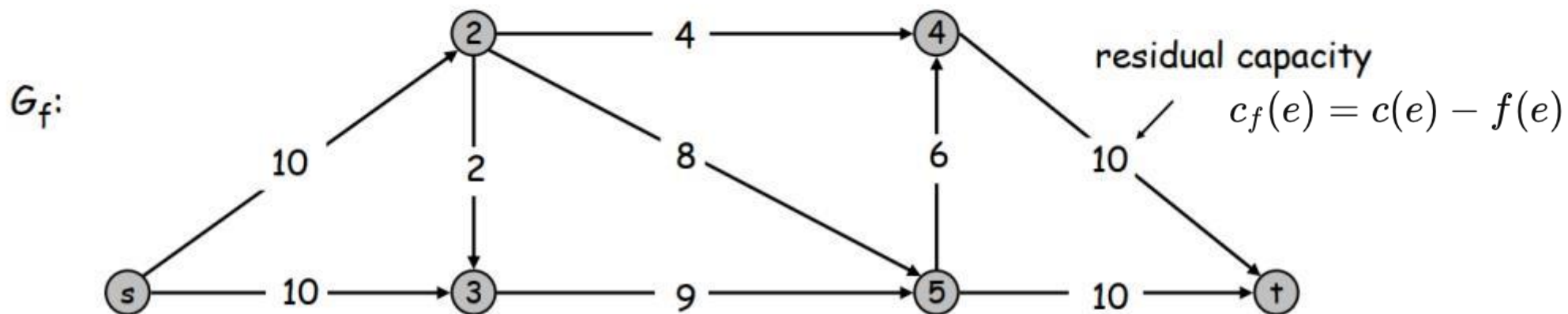




Ford-Fulkerson Algorithm

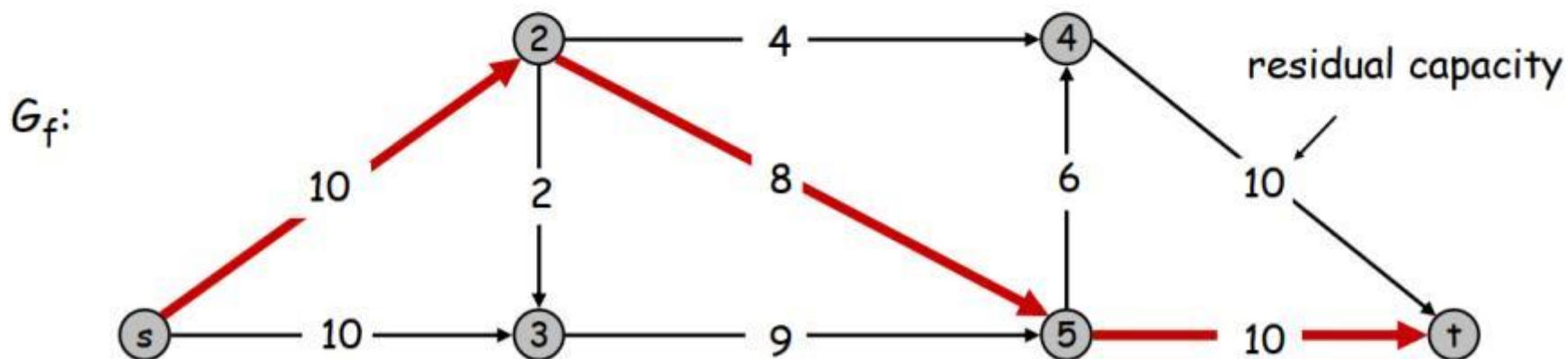
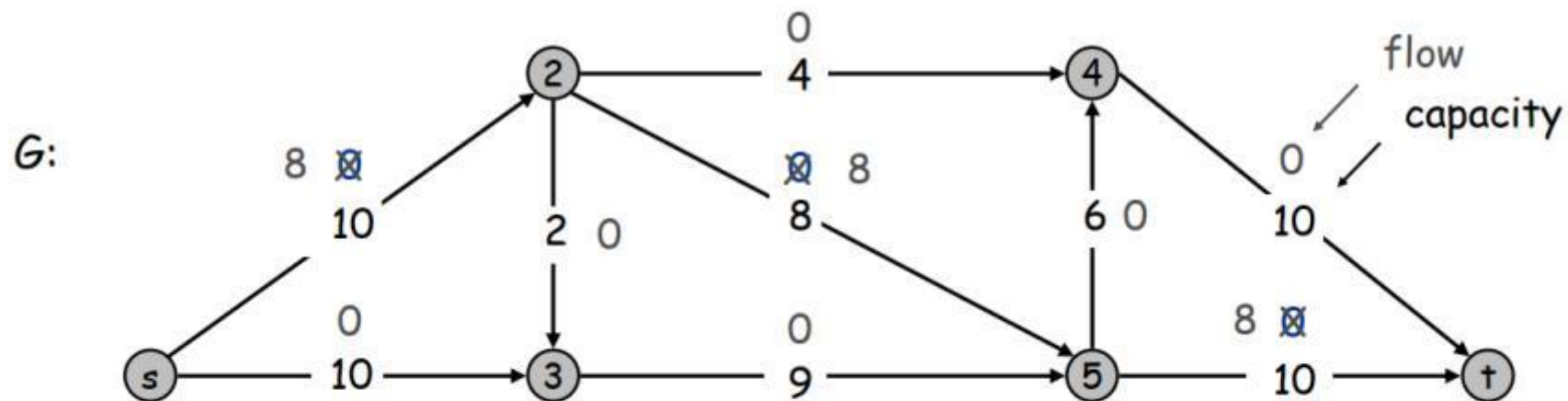


Flow value = 0



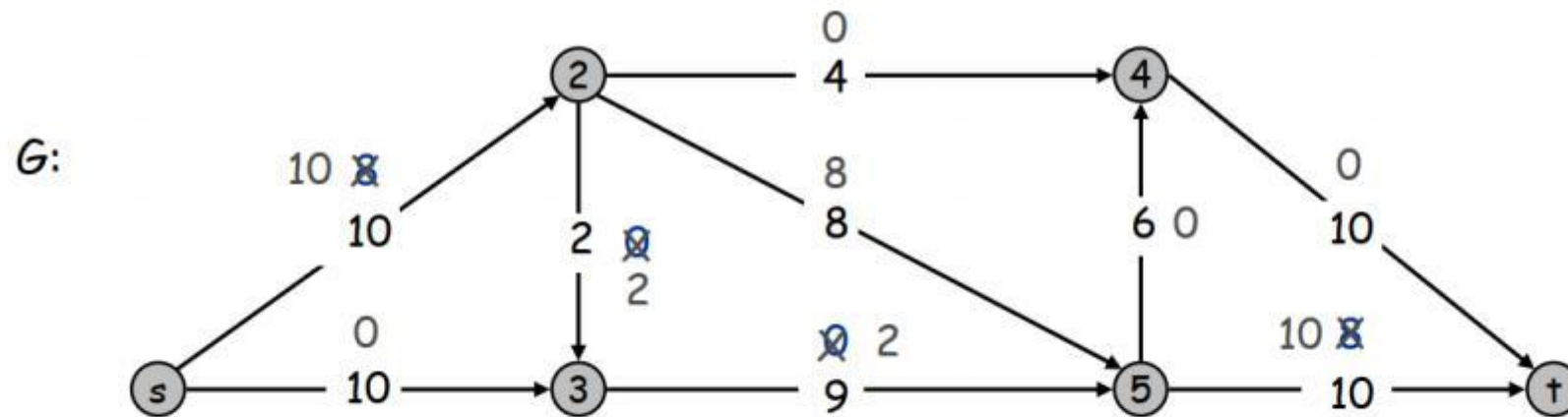


Ford-Fulkerson Algorithm

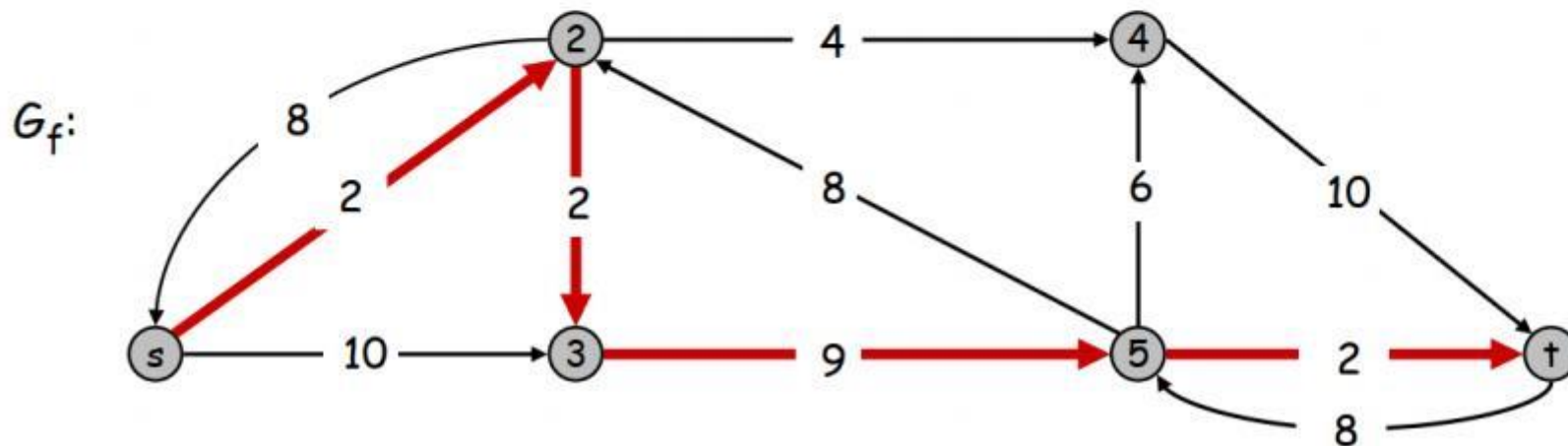




Ford-Fulkerson Algorithm

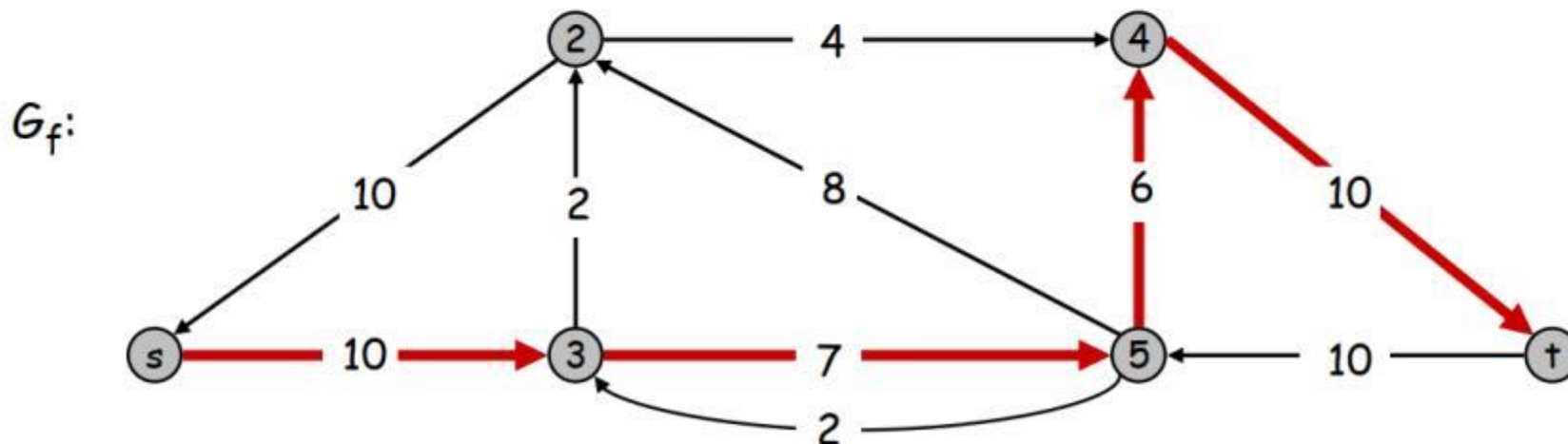
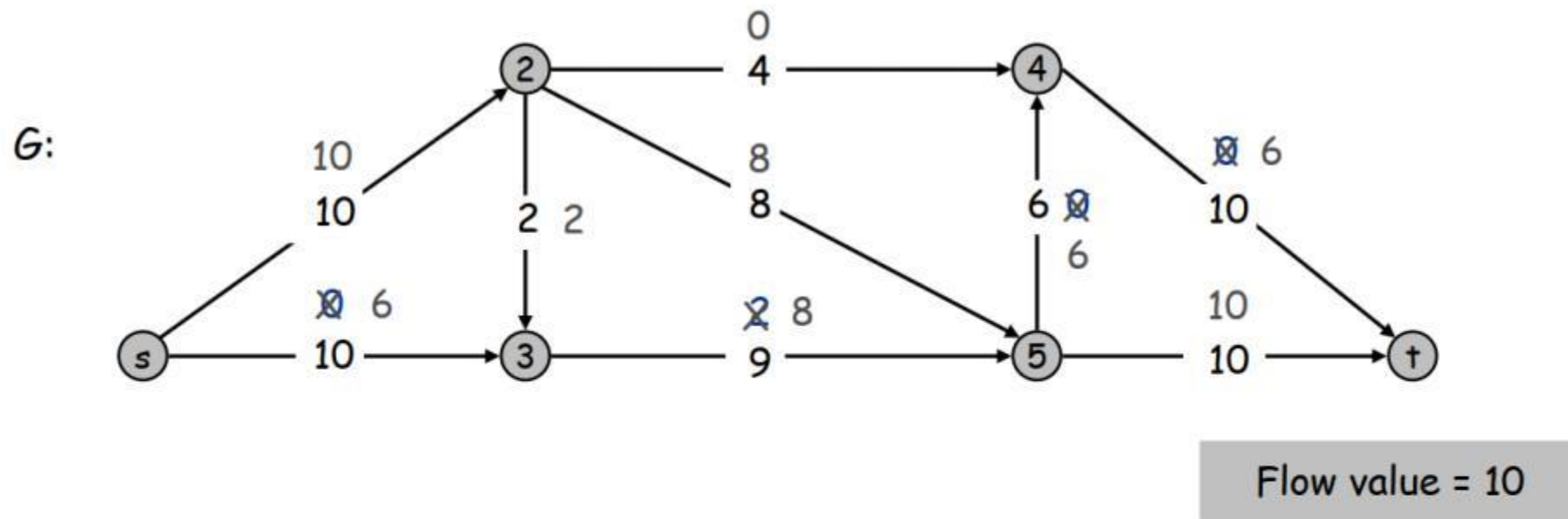


Flow value = 8



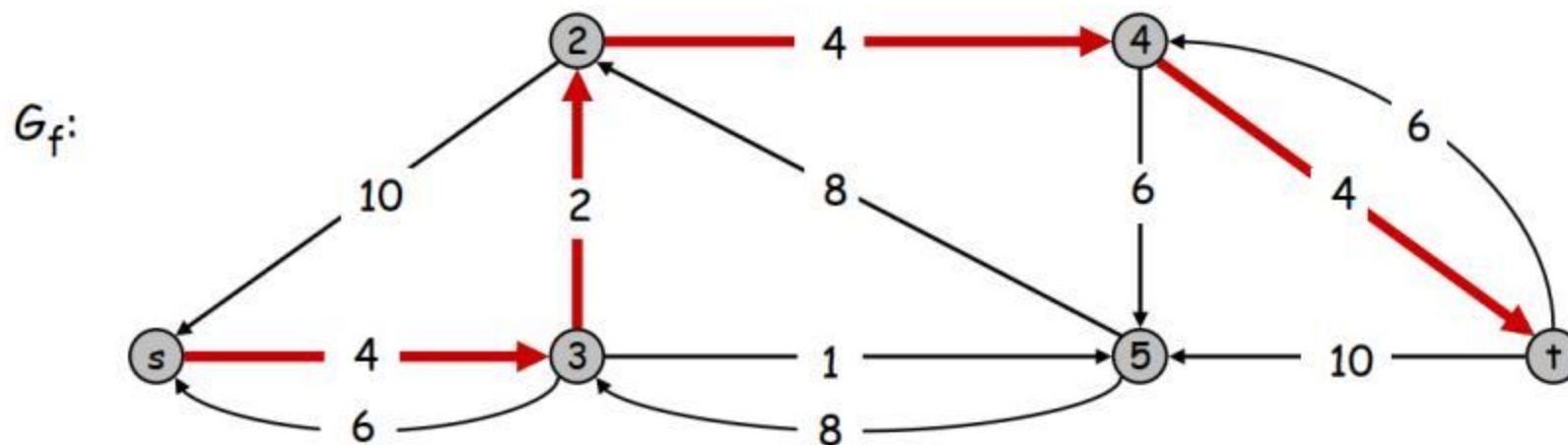
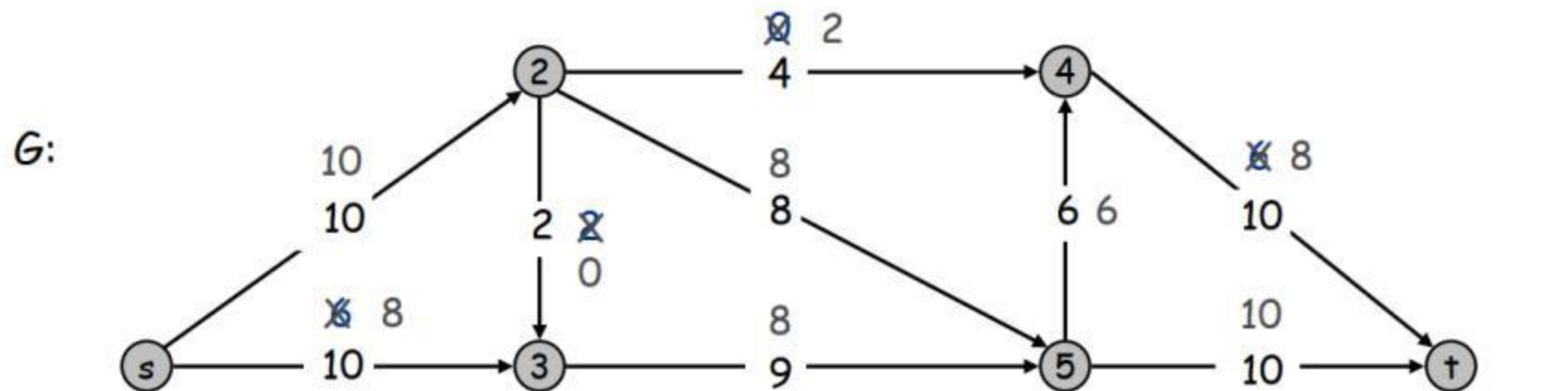


Ford-Fulkerson Algorithm



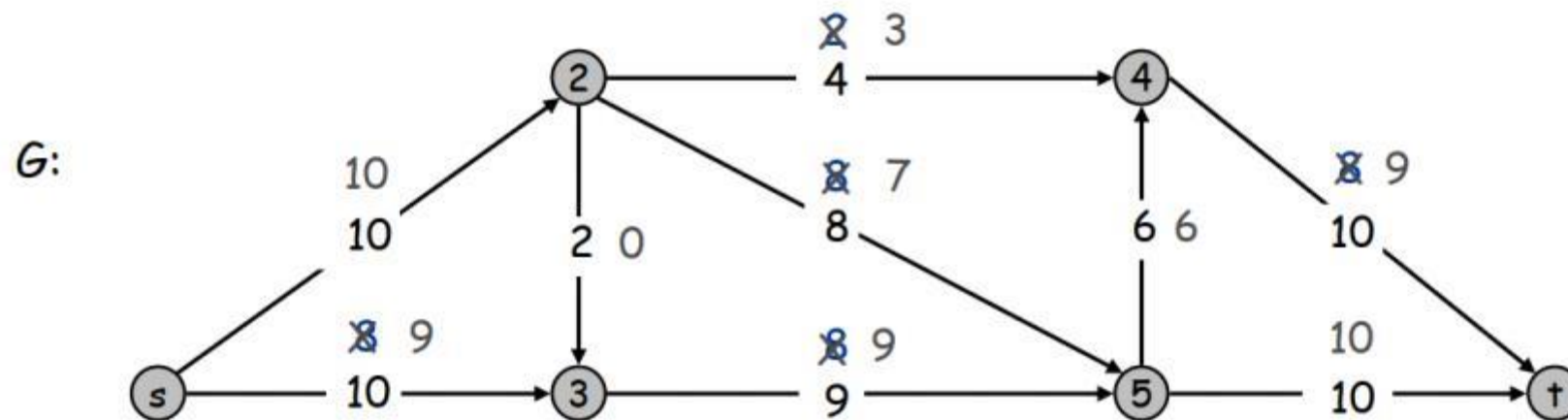


Ford-Fulkerson Algorithm

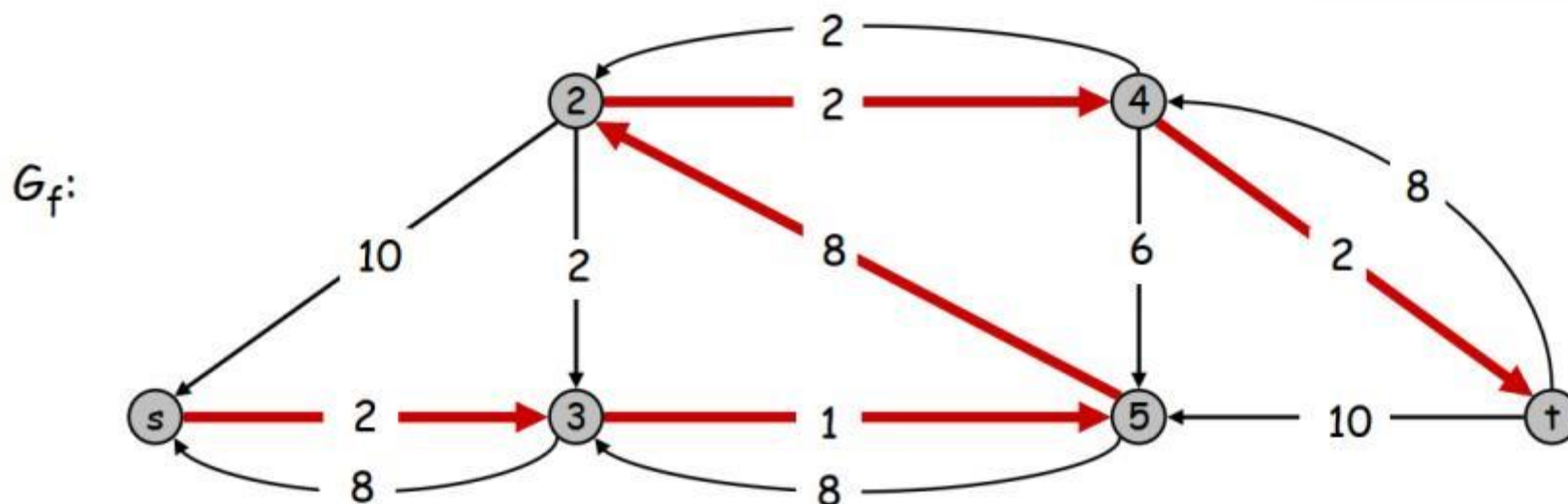




Ford-Fulkerson Algorithm

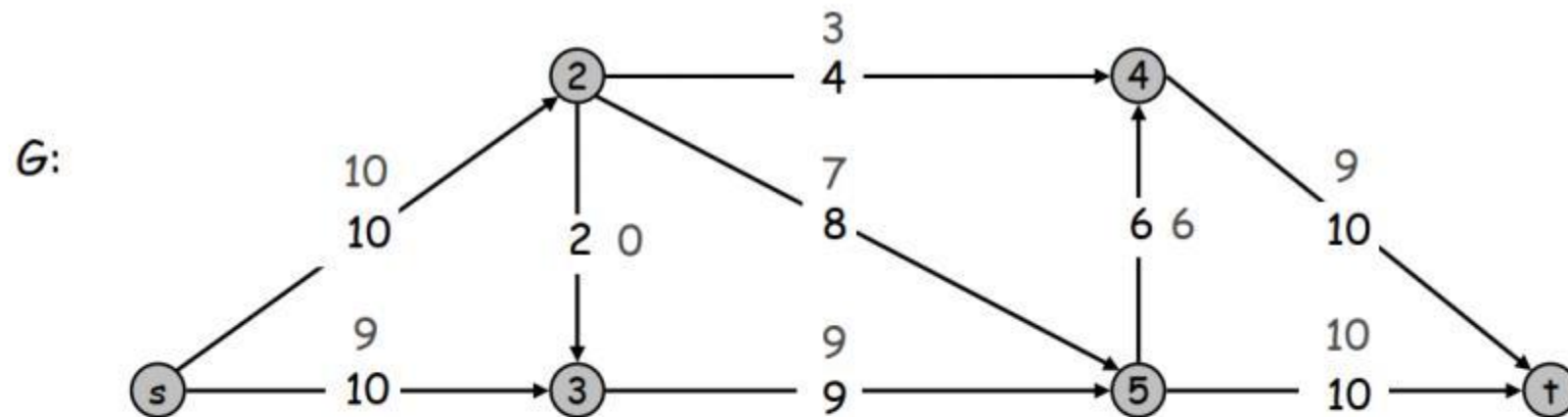


Flow value = 18





Ford-Fulkerson Algorithm



Flow value = 19

